Knapsack Comparison

0/1 Knapsack (Dynamic Programming) vs Fractional Knapsack (Greedy)

NAME: ADITHYA R
REGISTER NUMBER: 192311369

Domain: Design and Analysis of Algorithms
Techniques Compared: Dynamic Programming vs Greedy

1. Problem Statement

We are given 3 items with the following weights and values, and a knapsack of capacity W = 50:

Item	Weight	Value	Value/Weight
Item 1	10	60	6.0
Item 2	20	100	5.0
Item 3	30	120	4.0

Two variants are solved:

- 0/1 Knapsack: each item must be taken whole or not at all — solved optimally with Dynamic Programming.
- Fractional Knapsack: items may be split into fractions — solved optimally with a Greedy strategy.

2. 0/1 Knapsack — Dynamic Programming

2.1 Recurrence Relation

Let dp[i][w] denote the maximum value achievable using the first i items with knapsack capacity w. The recurrence is:

```
dp[0][w] = 0                                        for all w  (no items → no value)
dp[i][w] = dp[i-1][w]                                if weight[i] > w   (item cannot
fit)
dp[i][w] = max( dp[i-1][w],
                value[i] + dp[i-1][w - weight[i]] )   otherwise

Answer = dp[n][W]
```

Each cell represents a binary decision: exclude item i (carry forward dp[i-1][w]) or include item i (add its value and recurse on the reduced capacity w − weight[i]). Because every subproblem dp[i][w] is solved once and reused, this avoids the redundant recomputation that plain recursion would incur — the hallmark of dynamic programming applied to a problem with overlapping subproblems and optimal substructure.

2.2 DP Table (selected capacity columns)

The full table has 4 rows (items 0–3) × 51 columns (capacities 0–50). Representative columns are shown below; the final answer is read from the bottom-right cell, dp[3][50].

Items \ w	0	10	20	30	40	50
∅ (none)	0	0	0	0	0	0
+Item 1	0	60	60	60	60	60

(w=10,v=60)						
+Item 2 (w=20,v=100)	0	60	100	160	160	160
+Item 3 (w=30,v=120)	0	60	100	160	180	**220**

2.3 Backtracking the Chosen Items

Starting at dp[3][50] = 220, trace upward: since dp[3][50] ≠ dp[2][50] (220 ≠ 160), Item 3 was included — move to dp[2][50-30] = dp[2][20] = 100. Since dp[2][20] ≠ dp[1][20] (100 ≠ 60), Item 2 was included — move to dp[1][20-20] = dp[1][0] = 0. Since dp[1][0] = dp[0][0], Item 1 was not included. This confirms the optimal 0/1 selection:

Item Selected	Weight	Value	Fraction Taken
Item 2	20	100	100% (whole)
Item 3	30	120	100% (whole)
Total	**50 / 50**	**220**	**—**

0/1 Knapsack Optimal Value = 220, achieved by taking Item 2 and Item 3 whole, which exactly fills the capacity (20 + 30 = 50). Item 1, despite having the best value/weight ratio (6.0), is left out entirely because no combination that includes it can beat 220 under the all-or-nothing constraint.

3. Fractional Knapsack — Greedy Algorithm

3.1 Greedy Strategy

The greedy strategy for fractional knapsack is: compute the value-to-weight ratio of every item, sort items in decreasing order of ratio, and fill the knapsack by taking as much as possible of the highest-ratio item first, moving to the next item only when capacity remains and the current item is exhausted. If the remaining capacity is less than the next item's full weight, take only the fraction that fits.

```
function FRACTIONAL-KNAPSACK(items, W):
    sort items by (value / weight) in DESCENDING order
    totalValue ← 0
    remaining ← W

    for each item in sorted items:
        if item.weight <= remaining:
            take item fully
            totalValue ← totalValue + item.value
            remaining ← remaining - item.weight
        else:
```

```
            fraction ← remaining / item.weight
            totalValue ← totalValue + item.value * fraction
            remaining ← 0
            break                          // knapsack full

    return totalValue
```

3.2 Step-by-Step Execution

Items sorted by ratio (descending): Item 1 (6.0) → Item 2 (5.0) → Item 3 (4.0). Capacity available = 50.

1. Take Item 1 fully: weight 10 ≤ 50. Add value 60. Remaining capacity = 50 − 10 = 40.
2. Take Item 2 fully: weight 20 ≤ 40. Add value 100. Remaining capacity = 40 − 20 = 20.
3. Item 3 has weight 30 > remaining capacity 20 — take a fraction: fraction = 20/30 = 0.667. Add value 120 × 0.667 = 80. Remaining capacity = 0.
4. Capacity exhausted — stop.

Item	Ratio	Weight Taken	Fraction	Value Gained
Item 1	6.0	10	100%	60
Item 2	5.0	20	100%	100
Item 3	4.0	20	66.7%	80
Total	**—**	**50 / 50**	**—**	**240**

Fractional Knapsack Optimal Value = 240, achieved by taking Item 1 fully, Item 2 fully, and exactly 2/3 of Item 3.

4. Comparison of Solutions

Aspect	0/1 Knapsack (DP)	Fractional Knapsack (Greedy)
Optimal Value	**220**	**240**
Items Chosen	Item 2 (100%) + Item 3 (100%)	Item 1 (100%) + Item 2 (100%) + Item 3 (66.7%)
Item Divisibility	Not allowed — whole items only	Allowed — items can be split
Approach	Exhaustive-but-efficient table filling; considers all include/exclude combinations	Locally optimal choice at each step (highest ratio first), never reconsidered
Guaranteed Optimal?	Yes, always, for the 0/1 variant	Yes, always, for the fractional variant

The two answers, 220 and 240, are both correct — they are optimal for two different problems, not two different answers to the same problem. The gap of 20 exists purely because fractional knapsack is permitted to

take a partial slice of Item 3, which 0/1 knapsack cannot. This illustrates a general fact: relaxing a constraint (divisibility) can only improve or maintain the optimal value, never worsen it, because every valid 0/1 solution is also a valid fractional solution (with fractions restricted to 0 or 1).

5. Strengths and Limitations

5.1 Dynamic Programming (0/1 Knapsack)

- Strength: Always produces the true optimal solution for the 0/1 variant, because it systematically evaluates every relevant include/exclude combination via the recurrence, without missing any case.
- Strength: Handles complex constraints (indivisible items, discrete resource allocation) that greedy cannot handle correctly.
- Limitation: Time and space complexity scale with capacity W, not just the number of items — this is pseudo-polynomial, not polynomial in the input size (since W can be exponentially large relative to the number of bits used to encode it).
- Limitation: Impractical for very large capacities (e.g., W in the millions) even though n is small.

5.2 Greedy (Fractional Knapsack)

- Strength: Extremely fast — dominated by the O(n log n) sort, with no need for a capacity-sized table.
- Strength: Provably optimal for the fractional variant, backed by an exchange-argument proof (any solution not following the ratio order can be improved by shifting weight toward the higher-ratio item).
- Limitation: The same greedy ratio-based rule does NOT produce the optimal answer for 0/1 knapsack — applying it there would greedily take Item 1 (60) then Item 2 (100) for weight 30, leaving 20 capacity unusable by Item 3 (needs 30), giving only 160 total — worse than the true 0/1 optimum of 220.
- Limitation: Only meaningful when items are genuinely divisible (e.g., grain, liquids, currency) — not applicable to physically indivisible items (e.g., a single machine, a whole job).

This last limitation is worth emphasizing with the numbers already computed: naively running the greedy ratio rule on the 0/1 version of this exact problem would select Item 1 and Item 2 (weights 10+20=30, value 160) and then be unable to fit any part of Item 3, yielding only 160 — a full 60 below the DP optimum of 220. Greedy's locally optimal choices do not backtrack or reconsider, so an early greedy commitment can permanently block a better global combination; this is precisely why greedy is unsafe for 0/1 knapsack despite working perfectly for the fractional version.

6. Complexity Analysis

Method	Time Complexity	Space Complexity
0/1 Knapsack (DP, full table)	O(n × W)	O(n × W)
0/1 Knapsack (DP, rolling array)	O(n × W)	O(W)
Fractional Knapsack (Greedy)	O(n log n) (sorting-dominated)	O(1) extra (O(n) to store items)

For this problem instance, n = 3 and W = 50, so the DP table has 4 × 51 = 204 cells — trivial here, but the same problem with W = 10,000,000 would require over 40 million cells even though n is unchanged, illustrating the pseudo-polynomial blow-up. The greedy algorithm's cost is entirely determined by n (via the O(n log n) sort) and is completely insensitive to the magnitude of W, since it never builds a capacity-indexed table — it only ever performs a single pass over the sorted item list.

7. When Greedy Is Optimal vs. When DP Is Necessary

7.1 Why Greedy Works for Fractional Knapsack

Fractional knapsack exhibits the greedy-choice property: at every step, taking as much as possible of the currently best-ratio item is guaranteed to be part of some optimal solution. This can be proven by an exchange argument — if an optimal solution took less than the maximum possible of the best-ratio item while taking more of a lower-ratio item, swapping weight from the lower-ratio item to the higher-ratio item (allowed, since items are divisible) strictly increases total value without exceeding capacity, contradicting optimality. Because this exchange is always possible when items can be split, greedy never needs to reconsider an earlier choice, and a single forward pass suffices.

7.2 Why Greedy Fails for 0/1 Knapsack

0/1 knapsack does not have the greedy-choice property, because the exchange argument breaks down: items cannot be partially swapped, so committing fully to a high-ratio item may consume exactly the capacity needed by a combination of other items that would have produced a higher total value (as shown in Section 5.2, where greedy's 160 is beaten by DP's 220). The problem does, however, have optimal substructure (the optimal solution for capacity w using i items is built from optimal solutions to smaller subproblems), which is exactly the property dynamic programming is designed to exploit, since it considers both the include and exclude branch at every state rather than committing irrevocably.

7.3 General Decision Rule

- Use Greedy when: the problem has both optimal substructure AND the greedy-choice property (each locally optimal step is provably safe) — e.g., fractional knapsack, activity selection, Huffman coding, minimum spanning tree (Kruskal/Prim).
- Use DP when: the problem has optimal substructure but overlapping subproblems without a provably safe greedy rule — i.e., the locally best choice can block a better global solution, requiring the algorithm to explore/compare multiple choices via a recurrence — e.g., 0/1 knapsack, longest common subsequence, matrix chain multiplication.
- A quick diagnostic: try to construct a counter-example where the greedy rule's first choice provably prevents the true optimum. If one exists (as it does here — Item 1 blocks the better Item 2+3 combination), greedy is unsafe and DP (or another exhaustive-but-efficient method) is required.

8. Insights on Method Selection

- Optimality vs. speed trade-off: Greedy is faster and simpler but only safe when the problem's structure guarantees that local optimality implies global optimality; DP is slower/heavier but safe for a broader class of problems, including this one.
- Same data, different guarantees: This example proves that identical input data can yield different optimal values (220 vs 240) purely because of a change in problem constraints (divisibility) — method choice must be driven by the problem definition, not just the data.
- Cost of the wrong tool: Applying greedy logic to 0/1 knapsack silently produces a plausible-looking but suboptimal answer (160 instead of 220) with no error or warning — the algorithm still terminates and returns a number, making this a dangerous class of bug in real systems (e.g., resource allocation, budgeting, cargo loading) if the divisibility assumption is not checked first.
- Practical guideline: If items/resources in the real-world problem can genuinely be divided (currency, weight of bulk goods, time slices), reach for greedy for its speed. If they are discrete and indivisible (whole objects, whole jobs, whole investments), use DP, and be mindful of capacity size when choosing between full-table and rolling-array DP implementations for memory efficiency.

9. Conclusion

For the given instance (weights [10, 20, 30], values [60, 100, 120], capacity 50), 0/1 Knapsack via Dynamic Programming yields an optimal value of 220 (items 2 and 3 taken whole), while Fractional Knapsack via Greedy yields an optimal value of 240 (items 1 and 2 whole, plus two-thirds of item 3). Both results are correct for their respective problem definitions. The comparison demonstrates a central lesson in algorithm design: greedy algorithms are optimal only when the problem provably satisfies the greedy-choice property, and dynamic programming remains necessary whenever a locally best decision can foreclose a better global outcome — exactly the situation created by the indivisibility constraint in 0/1 knapsack.